Lab. Session 1. Introduction to Natural Language Processing. Tokenization and PoS Tagging (*NLTK*).

Goal

This practical activity focuses on creating a text tokenizer for the English language, using regular expressions, and constraining several specific linguistic rules. Nonetheless, it provides the required knowledge to use the NLTK library for corpus processing and Part-of-Speech (PoS) tagging. Moreover,

Bibliography

Regular Expressions:
http://docs.python.org/library/re.html
http://docs.python.org/howto/regex.html#regex-howto

NLTK Library
http://www.nltk.org/
https://www.nltk.org/book/

Natural Language Processing

Natural language, while effortlessly processed by humans early on, presents extraordinary challenges for computational systems. The apparent simplicity humans communicate—engaging in conversations, understanding context, interpreting tone, and adapting to linguistic variations—masks the incredible complexity underlying language processing. Prior to the development of large language models like ChatGPT, machines significantly lagged human capabilities in language understanding and processing. Natural Language Processing (NLP) involves analyzing language at multiple levels to enable computers to understand and process human language effectively. The levels of analysis in NLP can be broadly categorized into the following:

- Phonetic: Studies the sound of speech through which we perceive meaning.
- **Morphological and Lexical:** Studies internal structure of words and their formation. At this level, individual words are analyzed and processed.
- Syntactic: Studies the combination of words and phrases to construct sentences.
- Semantics: Studies the meaning of each word, phrase, and sentence.
- Pragmatic: Studies the role of context in the understanding of the meaning.

The **Morphological** and **Lexical** task involves:

1. Tokenization: Breaking text into individual words or tokens
2. Part-of-speech tagging: Identifying word categories (nouns, verbs, adjectives, etc.)
3. Lemmatization and Stemming: Reducing words to their base or root forms, respectively.

What is a Tokenizer?

Tokenizers are fundamental components used in NLP **to segment text into smaller units, known as tokens**. These tokens can be words, subwords, characters, or other meaningful units, depending on the specific tokenization technique. Tokenization serves as a fundamental preprocessing step in various NLP tasks, including text classification, sentiment analysis, machine translation, and information retrieval.

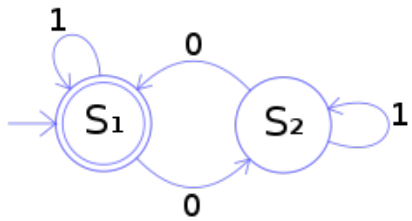
Main Types:

- Word Tokenizers: Split text into words ("*I love data" → ["I", "love", "data"]*)
- Subword Tokenizers: Break words into meaningful subunits (*"playing" → ["play", "##ing"]*)
- Character Tokenizers: Split text into individual characters
- Whitespace Tokenizers: Simple splitting on spaces

How to build a Tokenizer?

To construct robust and efficient tokenizers, we can leverage the theory of formal languages. Specifically, regular languages and finite-state automata provide a powerful framework for defining and recognizing patterns in text.

- Finite-State Automata (FSA): FSAs are computational mathematical models that can be used to recognize patterns in text. They consist of a finite set of states and a set of transitions between those states. By defining a specific set of states and transitions, we can create an FSA that recognizes a particular regular language.



- **Regular Expressions:** They are a concise and flexible way to represent patterns in text. They can be used to define the rules for tokenization. For instance, a regular expression can specify the patterns for recognizing words, numbers, punctuation marks, or other token types.

$$(\backslash w+@\backslash w+(\backslash.\backslash w+)+)$$

Regular Expressions in Python

Regular expressions were first formally described in 1951 by mathematician Stephen Cole Kleene, building on the concept of regular events. Their widespread popularity began in 1968, primarily due to their effectiveness in two key areas:

- **Text Pattern Matching:** Regular expressions are invaluable for searching and extracting specific patterns within text strings.
- **Lexical Analysis**: They play a crucial role in the lexical analysis phase of compilers, breaking down source code into meaningful tokens.

Basic Regular Expression Patterns

The simplest kind of regular expression is a sequence of simple characters; putting **concatenation characters** in sequence is called concatenation. To search for *datascience*, we type r'*datascience'*.

Regex	Example Patterns Matched
`r'datascience'`	*I really want to study **datascience***
`r'ugly'`	*The virus was exceptionally **ugly***
`r'!'`	*Computers are amazing**!***

EXAMPLE 1: Simple Regular Expressions

```
import re
s1 = "I really want to study datascience"
s2 = "The virus was exceptionally ugly"
s3 = "Computers are amazing!"
matching =re.search(r'datascience', s1)
print(matching.group())
matching =re.search(r'ugly', s2)
print(matching.group())
matching =re.search(r'!', s3)
print(matching.group())
```

Regular expressions are *case sensitive*; lower case *r's'* is distinct from upper case *r'S'* (*r's'* matches a lower-case *s* but not an upper-case *S*). This means that the pattern r'datascience' will not match the string *Datascience*. We can solve this problem with the use of the square braces *[* and *].* Regular expressions can be composed of **disjunction of characters** to match. For example, the pattern *r'[dD]'* matches patterns containing either *d* or *D.*

Regex	Example Patterns Matched
`r'[Dd]atascience'`	***Datascience*** *and datascience*
`r'[abc]'`	*The dog is on the s**a**nd*
`r'[0123456789]'`	*The **1**0% of students like programming*

EXAMPLE 2: Disjunction of Characters

```
s1 = "Datascience and datascience"
s2 = "The dog is on the sand"
s3 = "The 10% of students like programming"
matching =re.search(r'[dD]atascience', s1)
print(matching.group())
matching =re.search(r'[abc]', s2)
print(matching.group())
matching =re.search(r'[0123456789]', s3)
print(matching.group())
```

In cases where there is a well-defined sequence associated with a set of characters, the brackets can be used with the dash (-) to specify the ***range* of any one character** in a range. The pattern [2-5] specifies any one of the characters 2, 3, 4, or 5. The pattern [b-g] specifies one of the characters b, c, d, e, f, or g.

Regex	Match	Example Patterns Matched
`r'[A-Z]'`	An upper-case letter	*enrolled in the **N**LP course*
`r'[a-z]'`	A lower-case letter	***l**earning about language is great*
`r'[0-9]'`	A single digit	*only **1**2 lessons of the NLP*

EXAMPLE 3: Range of Characters

```
s1 = "enrolled in the NLP course"
s2 = "learning about language is great"
s3 = "only 12 lessons of the NLP"
matching=re.search(r'[A-Z]',s1)
print(matching.group())
matching=re.search(r'[a-z]',s2)
print(matching.group())
matching=re.search(r'[0-9]',s3)
print(matching.group())
```

The square braces can also be used to specify what a single character **cannot** be in the pattern, by use of the symbol ˆ. If the symbol ˆ is the first symbol after the open square brace [, the resulting pattern is negated. For example, the pattern *r'[ˆa]'* matches any single character (including special characters) except the character *a.*

Regex	Match	Example Patterns Matched
`r'[^A-Z]'`	not an upper-case letter	*E**n**rolled In tHe NLP course*
`r'[^Ss]'`	neither S nor s	*S**p**ring is sweet and safe*
`r'[^.]'`	not a period	*our professor Rosso*
`r'[e^]'`	either e or ˆ	*Look up ˆ now*
`r'a\^b'`	the pattern aˆb	*Look up **aˆb** now*

How can we define optional elements in the sequence, like an optional *m* in *programmers*? We cannot use the square brackets, because while they allow us to specify *m* or *M,* they do not allow us to define *m* or *nothing*. For this we use the question mark (***?***), which means the preceding character or *nothing.* One important character is the period (**.**), a wildcard expression that matches any single character.

Regex	Match	Example Patterns Matched
`r'programm?ers'`	Match with the preceding character or nothing	*The **programers** are busy*
`r'colou?r'`	color or colour	*True **colours** in the sky*
`r'beg.n'`	any character between beg and n	begin, beg'n, begun

EXAMPLE 4: Using Period (.) and ?

```
s1 = "The programmers are busy"
s2 = "True colours in the sky"
s3 = "The show begin at 8pm"
matching=re.search(r'programm?ers',s1)
print(matching.group())
matching=re.search(r'colou?rs',s2)
print(matching.group())
matching=re.search(r'beg.n',s3)
print(matching.group())
```

The symbol ***?*** as means **zero or one** instance of the previous character. However, it is not able to define one or more instances of the previous character or pattern. For example, consider the language which consists of strings that look like the following: *baa*!, *baaa!, baaaa!*, so on. This language consists of strings with a b, followed by at least two *a's*, followed by an *!*. The set of operators that allows us to define expressions s like "some Kleene * number of as" are based on the symbol *****, commonly called the **Kleene ***.

Sometimes it is annoying to have to write the regular expression for digits twice, so there is a shorter way to specify "at least one" of some character. This is the **Kleene +**, which means "one or more occurrences of the immediately preceding character or regular expression.

Regex	Match	Example Patterns Matched
`r'baa*!'`	Sequences which start with *ba* and end with *!*	*baa! baaaa! baaaa! ...*
`r'aa*'`	Sequences with one a followed by zero or more a's	*aa aaaaa aaaaaaa ...*
`r'[ab]*'`	Sequence with zero or more a's or b's	*ab aab baab, babba ...*
`r'[0-9]+`	All sequence of digit	*0 01 12 359 99999 ...*

EXAMPLE 5: Using Kleene * and Kleene+

```
s1 = "baa! baaaa! baaaa! ... "
s2 = "aa aaaaa aaaaaaa ... "
s3 = "ab aab baab, babba ... "
s4 = "0 01 12 359 99999 ... "
matching=re.search(r'baa*!',s1)
print(matching.group())
matching=re.search(r'aa*',s2)
print(matching.group())
matching=re.search(r'[ab]*',s3)
print(matching.group())
matching=re.search(r'[0-9]+',s4)
print(matching.group())
```

Also, a range of numbers can also be specified. So ***{n,m}*** specifies from ***n*** to ***m*** occurrences of the previous character or expression, and ***{n,}*** means at least ***n*** occurrences of the previous expression.

Regex	Example Patterns Matched
`r'{n}'`	*Exactly n occurrences of the previous char or expression*
`r'{n,}'`	*At least n occurrences of the previous char or expression*
`r'{n,m}'`	*From n to m occurrences of the previous char or expression*
`r'{,m}'`	*Up to m occurrences of the previous char or expression*

Anchors are special characters anchoring regular expressions to certain places in a string. The most common anchors are the symbols ˆ and **$**. The symbol ˆ matches the start of a line and the symbol **$** matches the end of a line. There are also two other anchors: **\b** matches a word boundary, and **\B** matches a non-word boundary.

Regex	Match	Example Patterns Matched
`r'^Python is'`	Lines starting with *Python is*	***Python is*** *easy*
`r'useful\.$'`	Lines ending with *useful.*	*Programming is* ***useful.***
`r'^Python is useful\.$'`	Lines exactly like *Python is useful.*	***Python is useful.***
`r'\bthe\b'`	The word *the*, but does not match the word others	*Students have* ***the*** *Test*
`r'\Bthe\B'`	The words that contain *the* but not at the beginning nether at the end	*Students have ano****the****r Test*

EXAMPLE 6: Anchors

```
s1 = "Python is easy"
s2 = "Programming is useful."
s3 = "Python is useful."
s4 = "Students have the Test"
s5 = "Students have another Test"
matching=re.search(r'^Python is',s1)
print(matching.group())
matching=re.search(r'useful\.$',s2)
print(matching.group())
matching=re.search(r'^Python is useful\.$', s3)
print(matching.group())
matching=re.search(r'\bthe\b',s4)
print(matching.group())
matching=re.search(r'\Bthe\B', s5)
print(matching.group())
```

The are some aliases for common ranges, which can be used to save typing.

Regex	Expansion	Match	Example Patterns Matched
`r'\d'`	`[0-9]`	any digit	*The **3**rd year of DS*
`r'\D'`	`[^0-9]`	any non-digit	***N**atural language*
`r'\w'`	`[a-zA-Z0-9_]`	any alphanumeric and underscore	***m**any years of I+D+I*
`r'\W'`	`[^\w]`	a non-alphanumeric	*Great**!**!!!!*
`r'\s'`	`[ \r\t\n\f]`	whitespace (space, tab)	*Machine_intelligent*
`r'\S'`	`[^\s]`	Non-whitespace	***=**=?*

Certain special characters are referred to by special notation based on the backslash symbol \. The most common of these are the newline character **\n** and the tab character **\t**. To refer to characters that are special themselves (like., *, [, and \), precede them with a backslash, (i.e., \., *, \[, and \\).

Regex	Match	Example Patterns Matched
`r'\*'`	an asterisk *	D*A*T*A*S
`r'\.'`	a period .	Prof**.** Rosso
`r'\?'`	a question mark	Why Data Science**?**
`r'\n'`	a new line	
`r'\t'`	a tab	

Regular expressions offer the **OR operator**, denoted by the vertical bar symbol (**|**), for pattern matching that enables the specification of alternative patterns. This operator functions as a logical OR, allowing you to match one pattern or another within your search criteria. When the regex engine encounters the | operator, it attempts to match the pattern on either side of it, effectively creating a choice between multiple alternatives. For example, in the

pattern *'cat|dog',* the regex will match either *'cat' or 'dog'* in the sequence. The OR operator becomes particularly powerful when combined with groups, as in the pattern *'(black|white) is (cat|dog)*', which can match combinations like *'cat is white'*, *dog is black'*, *'cat is black'*, or *'dog is white'*. The regex engine evaluates OR patterns from left to right. It stops at the first successful match, making it essential to order your alternatives appropriately, typically from most specific to least specific, to ensure the desired matching behaviour.

Regex	Match	Example Patterns Matched
`r'cat|dog'`	Words cat or doc	The **dog** is black
`r'(cat|dog)is (black|white).'`		The **cat is white**

Regular expressions can be grouped to: (i) organize and define precedence and (ii) assess subpatterns within the expression.

EXAMPLE 7: Groups

```
pattern = "(\w+)@((\w+\.)+(com|org|net|edu|es))"
matching = re.search(pattern,"student@upvnet.upv.es")
total_matches = len(matching.groups())
print("The number of matched groups is:", total_matches)
print("group 1",matching.group(1))
print("group 2", matching.group(2))
print("group 3",matching.group(3))
print("group 4",matching.group(4))
```

Named Groups: Named groups allow you to assign a name to a capturing group using the syntax **(?P<name>pattern)**. This makes

it easier to reference matched groups by name rather than by number. You can later reference these groups using the assigned name.

EXAMPLE 8: Named Groups

```
pattern =
"(?P<user>\w+)@(?P<fulldomain>(?P<middledomain>\w+\.)+(?P<lastdomain>com|org
|net|edu|es))"
matching = re.search(pattern,"student@upvnet.upv.es")
total_matches = len(matching.groups())
print("The number of matched groups is:", total_matches)

print("group user",matching.group("user"))
print("group fulldomain", matching.group("fulldomain"))
print("group middledomain",matching.group("middledomain"))
print("group lastdomain",matching.group("lastdomain"))
```

Non-Capturing Groups: Non-capturing groups, denoted by **(?:pattern),** allow you to group patterns together without creating a numbered reference. They are useful when you need to group elements but do not need to extract or reference that group later.

EXAMPLE 9: Non-Capturing Groups

```
pattern =
"(?P<user>\w+)@(?P<fulldomain>(?:\w+\.)+(?P<lastdomain>com|org|net|edu|es))"
matching = re.search(pattern,"student@upvnet.upv.es")
total_matches = len(matching.groups())
print("The number of matched groups is:", total_matches)

print("group user",matching.group("user"))
print("group fulldomain", matching.group("fulldomain"))
print("group lastdomain",matching.group("lastdomain"))
```

Back References: Back references let you match the same text matched by a previous capturing group. You can reference numbered groups with \1, \2, etc., or named groups with (?P=name).

EXAMPLE 10: Back References

```
#Back reference using named groups
pattern= r"(?P<word>\w+)\s+(?P=word)"
matching = re.search(pattern,"hello hello")
print(matching.group("word"))

#Back reference using unnamed groups
pattern= r"(\w+)\s+(\1)"
matching = re.search(pattern,"hello hello")
print(matching.group(1))
```

Regex offers two main methods to detect the patterns in the sequence, ***search*** and ***match***. The ***search*** method looks for a

pattern anywhere on the string and the ***match*** looks for a match to the pattern starting from the beginning of the sequence. Both methods return a "*match object*" if the pattern is found, and **None** in other cases. In the example below, the **search** method finds the pattern because it looks for a match anywhere in the sequence, while the match method does not, as it only checks for a match at the beginning of the sequence.

EXAMPLE 11: Methods *'search'* and *'match'*

```
pattern= r"(?P<word>\w+)\s+(?P=word)"
matching = re.search(pattern,"Only saying hello hello")
print(matching)

matching = re.match(pattern,"Only saying hello hello")
print(matching)
```

The ***"match object"*** *has the following methods to access the patterns matched.*

Method	Description
`group()`	Returns the string matching the regex
`start()`	Returns the starting position of the match
`end()`	Returns the end position of the match
`span()`	Returns a tuple containing (start, end) positions

In Python, the regular expressions can be compiled using compile(), the regular expression engine processes and optimizes the pattern once rather than each time the pattern is used. It is beneficial when you need to apply the same pattern multiple times. Compiling a regular expression allows you to separate the definition of the regex pattern from its usage, making the code more organized and

readable, especially for complex expressions. Once compiled, the regex object can be reused to perform multiple operations (such as search(), match(), finditer (), or findall()) without needing to recompile the pattern each time.

Method	Description
`match ()`	Determines if the regex matches the beginning of the string
`search ()`	Scans the string, looking for any place where it matches the regex
`findall ()`	Finds all substrings where the regex matches and returns them as a list
`finditer ()`	Finds all substrings where the regex matches and returns them as an iterator

There are some flags that can be used when a regular expression is compiled.

Flag	Description
`DOTALL, (S)`	Allows (.) to match any character, including \n
`IGNORECASE, (I)`	Ignores the use of upper- and lower-case letters
`MULTILINE, (M)`	Allows matching to be done on multiple lines ^ and $
`VERBOSE, (X)`	Allows writing comments inside of the regular expressions

EXAMPLE 12: Methods and Flags in Compiled Regular Expressions

```
text = """Hello World!
This is a test.
HELLO again."""

# Compile a regex pattern with multiple flags: IGNORECASE and MULTILINE
pattern = re.compile(r"^hello", re.IGNORECASE | re.MULTILINE)

# Checks if the pattern matches the beginning of the string
match_result = pattern.match(text)

# Finds the first occurrence of the pattern anywhere in the string
search_result = pattern.search(text)

# Finds all substrings matching the pattern and returns them as a list
findall_result = pattern.findall(text)

# Returns an iterator of match objects for all matches in the string
finditer_result = pattern.finditer(text)
for match in finditer_result:
    print(f"finditer(): Found '{match.group()}' at position {match.start()}")

# Using DOTALL flag to match across newlines
dotall_pattern = re.compile(r"Hello.*again", re.DOTALL)
dotall_match = dotall_pattern.search(text)
if dotall_match:
    print(f"DOTALL flag: Found '{dotall_match.group()}' across newlines.")

# Using VERBOSE flag to write readable regular expressions with comments
verbose_pattern = re.compile(r"""
    ^ # start of the line
    hello # match 'hello'
""", re.IGNORECASE | re.VERBOSE | re.MULTILINE)

verbose_search = verbose_pattern.search(text)
if verbose_search:
    print(f"VERBOSE flag: Found '{verbose_search.group()}' with comments.")
```

Modifying sequences based on regular expressions

An important use of regular expressions is in substitutions, for that purpose the regular expression in Python provides the methods:

Flag	Description
`split()`	Splits the string into a list, splitting it where it matches the regular expression
`sub()`	Finds all substrings where the regex matches and replaces them with a different string
`subn()`	Does the same as sub() but returns a new string and the number of replacements

EXAMPLE 13: Modifying Sequence with Regular Expressions

```
pattern = re.compile(r",")
result = pattern.split("apple,banana,orange")
print(result)

pattern = re.compile(r"\d+")
result = pattern.sub("number", "hello 123 world 456")
print(result)

pattern = re.compile(r"\d+")
result, count = pattern.subn("number", "hello 123 world 456")
print(result)
print(count)
```

Natural Language Toolkit (NLTK Python 3.x)

NLTK stands for Natural Language Toolkit. It is a comprehensive library for natural language processing tasks in Python. NLTK provides easy-to-use interfaces to over 50 corpora and lexical resources, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning.

NLTK Functionalities

- Corpora: It provides access to numerous text corpora and lexical resources in many languages and domains.
- Tokenization: It offers tokenization tools which are a fundamental preprocessing step in many NLP tasks.
- **Part-of-Speech (PoS) Tagging:** It includes tools for part-of-speech tagging, which assigns grammatical categories (e.g., noun, verb, adjective) to words in a sentence.
- Parsing: It provides parsers for syntactic analysis of sentences, including constituency parsing and dependency parsing.
- Stemming and Lemmatization: It offers algorithms for stemming and lemmatization, which are techniques used to reduce words to their base forms.
- Named Entity Recognition (NER): It includes tools for named entity recognition, which identifies and classifies named entities (e.g., persons, organizations, locations) in text.
- Sentiment Analysis: It provides tools and resources for sentiment analysis, which aims to determine the sentiment or opinion expressed in text.

Installing NLTK

- Install the source code using pip or conda

```
!pip install nltk
!conda install anaconda::nltk
```

- Download the corpora and lexical resources
  - The NLTK allows you to interactively select the datasets and resources you want to download. You can pass the identifier of the resource you want to install or all for installing all resources

```
import nltk
nltk.download(), nltk.download('all')
```

POS tagging

Part-of-speech (POS) tagging is a fundamental task in natural language processing (NLP) that involves labeling each word in a sentence with its corresponding part of speech, such as noun, verb, adjective, adverb, etc.

"The/? young/? boy/? can/? open/? the/? can/? with/? his/? hands/? ./?"

"The/DET young/ADJ boy/NOUN can/MD open/VERB the/DET can/NOUN with/PREP his/PRON hands/NOUN ./PUNT"

We can use the trained models available in the NLTK or train models on a specific corpus annotated with PoS tags:

- Train the model with the training corpus.
- Validate the quality of the model in a validation corpus.
- Use the trained model for labelling untagged sentences.

Technical details about training the tagger models can be found in the jupyter notebook `LNR_LAB_S1_NLTK.ipynb`

ACTIVITIES

Activity 1:

Build a tokenizer based on regular expressions. Given a text and a regular expression, the tokenizer will split the text into tokens as defined in the regular expression and returns them in a list.

The tokenizer must satisfy the following constraints:

1. The symbols *( ) . , ' " ? ¿ ! ¡ ...;:%* are considered special tokens and need to be separated from each word.
2. Decimal numbers must be kept as a single token, e.g., 44,45 45.60 28,10%
3. Do not separate dates 12/12/90, 12-03-98, or times like 9:30 am.
4. URLs like http://www.colorin.com must be kept.
5. e-mails address like username@cdit.com must be kept.
6. Mentions of users (@user) and hashtags (#hashtag) must be maintained.
7. Acronyms must be kept, for example, USA, S.L., CC.OO., S.A., D., U.R.S.S., …
8. Emoticons must be recognized 💕👫🎎😊, etc.

Resources (in PoliformaT):

- Notebook: `LNR_LAB_S1_activity_1.ipynb`
- Test data: `input-tokenizer.json`, `input-tokenizer.txt`

Activity 2:

In this activity you have to build a HMM[1]-based POS tagger trained with the Brown corpus.

a) Processing of the Brown corpus annotated with PoS labels.

Process the corpus to transform the annotation from the original tags (472 tags) to a simplified set (80 tags). To perform this transformation, use the following criterion: all tags will be simplified to keep only the grammatical category information.

- Remove the prefix "FW-" of a tag
- Split the PoS tag by symbols '-', '+' , '$' and '*' keeping the left part of the tag. For instance "NP+IN" will be simplified as "NP"
- Notice the specific tag "--" denoting a dash symbol.

NOTE: To understand the meaning of the labels, please refer to the following links:
https://varieng.helsinki.fi/CoRD/corpora/BROWN/tags.html
http://korpus.uib.no/icame/manuals/BROWN/INDEX.HTM#bc6

Example of tags simplification:

ORIGINAL
[('The', 'AT'), ('Fulton', 'NP-TL'), ('County', 'NN-TL'), ('Grand', 'JJ-TL'), ('Jury', 'NN-TL'), ('said', 'VBD'), ('Friday', 'NR'), ('an', 'AT'), ('investigation', 'NN'), ('of', 'IN'), ("Atlanta's", 'NP$'), ('recent', 'JJ'), ('primary', 'NN'), ('election', 'NN'), ('produced', 'VBD'), ('``', '``'), ('no', 'AT'), ('evidence', 'NN'), ("""", """"), ('that', 'CS'), ('any', 'DTI'), ('irregularities', 'NNS'), ('took', 'VBD'), ('place', 'NN'), ('.', '.')]

[1] Hidden Markov Models.

SIMPLIFIED

[('The', 'AT'), ('Fulton', 'NP'), ('County', 'NN'), ('Grand', 'JJ'), ('Jury', 'NN'), ('said', 'VBD'), ('Friday', 'NR'), ('an', 'AT'), ('investigation', 'NN'), ('of', 'IN'), ("Atlanta's", 'NP'), ('recent', 'JJ'), ('primary', 'NN'), ('election', 'NN'), ('produced', 'VBD'), ('``', '``'), ('no', 'AT'), ('evidence', 'NN'), ("""", """"), ('that', 'CS'), ('any', 'DTI'), ('irregularities', 'NNS'), ('took', 'VBD'), ('place', 'NN'), ('.', '.')]

b) Training and evaluation of the PoS tagger

** Use the simplified version of corpus Brown obtained in the previous activity **.

- Divide the corpus (simplified version) into training (90% of the first sentences) and testing (the remaining 10%).
- Practice training the HMM tagger using the previously simplified training partition.
- Practice labelling the examples in the testing set using the trained tagger.
- Evaluate the performance (*accuracy*) of the trained tagger.
- Evaluate the accuracy per tag using the resulting confusion matrix.
- Search some wrong sentences.

Resources (in PoliformaT):

- Notebook: `LNR_LAB_S1_activity_2.ipynb`